História de usuário · Observabilidade

Instrumentar o fluxo de eventos de pagamento com métricas Micrometer

TIPO	COMPONENTE	STACK	REFERÊNCIA
História técnica	Consumidor de eventos de pagamento	Java 21 · Spring Boot 3.5 · Micrometer · Prometheus · Grafana	POC `pagamentos-metricas`

> **Como** time responsável pelo fluxo de pagamentos,
> **quero** métricas de volume, sucesso, rejeição, erro e latência por arranjo e por status em cada etapa do processamento (fila, banco, máquina de estado e filas de saída),
> **para** detectar degradação, falhas de integração e inconsistências da máquina de estado antes do impacto no cliente, e ter base para SLOs e alertas.

1. Contexto

O serviço consome eventos do Azure Service Bus. Cada evento traz um **arranjo** (PIX dinâmico, PIX estático, PIX automático, TED, TEF, boleto) e um **status** (pagamento iniciado, análise de fraude, fraude aprovada/reprovada, pagamento concluído/rejeitado/devolvido, cancelado). O processamento busca a operação no banco, valida a transição na máquina de estado, grava o novo status e publica em uma fila de saída. Quando a transição não é permitida a partir do estado salvo, o serviço grava uma **ocorrência de falha**.

Hoje não há visibilidade de quanto tempo cada etapa leva, de onde vêm os erros nem de quantas transições inválidas acontecem por arranjo.

1. Recebe evento	2. Valida domínio	3. Busca operação	4. Máquina de estado	5. Grava status	6. Publica
Service Bus `espera_fila` · `reentrega`	arranjo, status, fluxo `nao_mapeado`	banco `db.operacao`	transição válida? `transicao.invalida`	banco `db.operacao`	fila de saída `servicebus.envio`

O timer `pagamento.evento.processamento` envolve as etapas 1 a 6, do recebimento ao *complete/abandon*.

2. Princípios de desenho

- **Poucas métricas, muitas tags.** Uma métrica por conceito, com dimensões em tags. Não criar uma métrica por status ou por arranjo.
- **Timer já é counter.** Todo Timer publica `_count`, `_sum`, `_max` e buckets. Com a tag `outcome`, ele já responde quantas chegaram, quantas deram sucesso e quantas falharam. Não criar counters paralelos de "recebidos/sucesso/erro".
- **Counter só para eventos sem duração:** transição inválida, cenário não mapeado, reentrega.
- **Três resultados, não dois.** `outcome` = `success` | `rejected` | `error`. `rejected` é recusa de regra de negócio (transição inválida, não mapeado). `error` é falha técnica. Isso evita que um surto de eventos fora de ordem dispare alerta de indisponibilidade.

- **Cardinalidade controlada.** Tags só com domínio fechado (enums). Nunca usar `operacaoId`, `endToEndId`, CPF/CNPJ ou mensagem de exceção como tag.
- **Mesmas chaves de tag sempre.** O registry do Prometheus rejeita a mesma métrica com conjuntos de tags diferentes. Valor ausente vira `desconhecido` ou `none`; a tag nunca é omitida.
- **`error.type` por categoria** (`db`, `servicebus`, `timeout`, `validacao`, `transicao_invalida`, `nao_mapeado`, `desconhecido`), nunca o nome da classe da exceção.
- **Histograma com SLO buckets explícitos** via `management.metrics.distribution.slo`. Não usar `percentiles()` no cliente, porque não agrega entre pods. Não usar `publishPercentileHistogram`, que gera cerca de 70 buckets por série.
- **Fachada única** (`PagamentoMetrics`) concentra nomes e chaves. O código de negócio não chama `MeterRegistry` diretamente.
- **Tags comuns** (`application`, `env`, `region`) no registry via `management.metrics.tags.*`, não em cada métrica.

3. Catálogo de métricas

Métrica (Micrometer)	Tipo	Tags	O que responde
`pagamento.evento.processamento`	Timer	`arranjo`, `status_evento`, `outcome`, `error.type`	Vazão por arranjo e status; sucesso, rejeição e erro; latência ponta a ponta.
`pagamento.evento.espera_fila`	Timer	`arranjo`	Tempo entre o `enqueuedTime` da mensagem e o início do processamento (backlog).
`pagamento.db.operacao`	Timer	`operacao`, `outcome`, `error.type`	Latência e erro de banco por operação de negócio.
`pagamento.servicebus.envio`	Timer	`destino`, `arranjo`, `outcome`, `error.type`	Envios para as filas de saída: volume, falhas e latência por fila.
`pagamento.transicao.invalida`	Counter	`arranjo`, `estado_atual`, `status_evento`	Ocorrências de falha da máquina de estado e qual transição está quebrando.
`pagamento.evento.nao_mapeado`	Counter	`arranjo`, `motivo`	Eventos que o fluxo não sabe tratar.
`pagamento.evento.reentrega`	Counter	`arranjo`	Mensagens com `deliveryCount > 1`: sinal precoce de DLQ.

No Prometheus os nomes viram `pagamento_evento_processamento_seconds_{count,sum,max,bucket}`, `pagamento_transicao_invalida_total` etc. A tag `error.type` vira o label `error_type`.

Domínio de cada tag

Tag	Valores
`arranjo`	`pix_dinamico`, `pix_estatico`, `pix_automatico`, `ted`, `tef`, `boleto`, `desconhecido`
`status_evento`	`pagamento_iniciado`, `analise_fraude`, `fraude_aprovada`, `fraude_reprovada`, `pagamento_concluido`, `pagamento_rejeitado`, `pagamento_devolvido`, `cancelado`, `desconhecido`
`estado_atual`	`nova`, `iniciada`, `em_analise_fraude`, `fraude_aprovada`, `fraude_reprovada`, `concluida`, `rejeitada`, `devolvida`, `cancelada`

`outcome`	`success`, `rejected`, `error`
`error.type`	`none`, `db`, `servicebus`, `timeout`, `validacao`, `transicao_invalida`, `nao_mapeado`, `desconhecido`
`operacao`	`buscar_operacao`, `atualizar_status`, `gravar_ocorrencia`
`destino`	Nome lógico da fila de saída: `antifraude-solicitacao`, `pagamento-concluido`, `pagamento-rejeitado`, `pagamento-devolvido`, `pagamento-cancelado`, `pagamento-status`
`motivo`	`status_desconhecido`, `arranjo_desconhecido`, `fluxo_nao_suportado`, `payload_invalido`

> **Mapeamento outcome × Service Bus:** `success` e `rejected` → *complete* (a ocorrência já foi gravada). `error` → *abandon*, que gera reentrega e, após o limite, DLQ.

4. Implementação

4.1 Componentes

Classe	Responsabilidade
`metricas.Tags`	Chaves de tag como constantes e enums de valores: `Outcome`, `ErrorType`, `OperacaoDb`, `MotivoNaoMapeado`. Cada enum expõe `tag()` em minúsculas.
`metricas.PagamentoMetrics`	Fachada única. Constrói e registra os meters. Expõe `iniciar()`, `registrarProcessamento()`, `registrarEsperaFila()`, `medirDb()`, `medirEnvio()`, `transicaoInvalida()`, `naoMapeado()`, `reentrega()` e `classificar(Throwable)`.
`dominio.Arranjo`, `StatusEvento`, `EstadoOperacao`	Enums de domínio com `tag()` e `parse()` tolerante: valor desconhecido resulta em `Optional.empty()`, e a tag recebe `desconhecido`.
`dominio.ProcessadorEventoService`	Handler do consumidor. Inicia o timer, registra espera e reentrega, valida, chama banco e publisher via fachada, e fecha o timer em um único ponto com `outcome` e `error.type`.
Consumidor Service Bus	Extrai `getEnqueuedTime()` e `getDeliveryCount()` de `ServiceBusReceivedMessage`, chama o serviço e faz *complete* ou *abandon* conforme o `outcome`.

4.2 Fachada de métricas (trecho)

```
@Component
public class PagamentoMetrics {
    public static final String PROCESSAMENTO      = "pagamento.evento.processamento";
    public static final String ESPERA_FILA        = "pagamento.evento.espera_fila";
    public static final String DB_OPERACAO        = "pagamento.db.operacao";
    public static final String SB_ENVIO           = "pagamento.servicebus.envio";
    public static final String TRANSICAO_INVALIDA = "pagamento.transicao.invalida";
    public static final String NAO_MAPEADO        = "pagamento.evento.nao_mapeado";
    public static final String REENTREGA          = "pagamento.evento.reentrega";

    private final MeterRegistry registry;

    public Timer.Sample iniciar() { return Timer.start(registry); }

    public void registrarProcessamento(Timer.Sample s, String arranjo, String status,
                                       Outcome outcome, ErrorType erro) {
        s.stop(Timer.builder(PROCESSAMENTO)
                .tag(Tags.ARRANJO, arranjo).tag(Tags.STATUS_EVENTO, status)
                .tag(Tags.OUTCOME, outcome.tag()).tag(Tags.ERROR_TYPE, erro.tag())
                .register(registry));
    }

    public <T> T medirDb(OperacaoDb op, Supplier<T> chamada) {
        Timer.Sample s = Timer.start(registry);
        Outcome outcome = Outcome.SUCCESS; ErrorType erro = ErrorType.NONE;
        try { return chamada.get(); }
        catch (RuntimeException e) { outcome = Outcome.ERROR; erro = classificar(e); throw e; }
        finally {
            s.stop(Timer.builder(DB_OPERACAO).tag(Tags.OPERACAO, op.tag())
                    .tag(Tags.OUTCOME, outcome.tag()).tag(Tags.ERROR_TYPE, erro.tag())
                    .register(registry));
        }
    }
    // medirEnvio(destino, arranjo, Runnable) segue o mesmo padrão

    public void transicaoInvalida(String arranjo, String estadoAtual, String status) {
        Counter.builder(TRANSICAO_INVALIDA).tag(Tags.ARRANJO, arranjo)
                .tag(Tags.ESTADO_ATUAL, estadoAtual).tag(Tags.STATUS_EVENTO, status)
                .register(registry).increment();
    }

    public static ErrorType classificar(Throwable e) {   // categoria, nunca a classe
        if (e instanceof TimeoutDependenciaException) return ErrorType.TIMEOUT;
        if (e instanceof FalhaBancoException)         return ErrorType.DB;
        if (e instanceof FalhaServiceBusException)    return ErrorType.SERVICEBUS;
        return ErrorType.DESCONHECIDO;
    }
}
```

4.3 Uso no fluxo (trecho)

```
public Resultado processar(EventoPagamento evento) {
    Timer.Sample sample = metrics.iniciar();
    String arranjoTag = Arranjo.parse(evento.arranjo()).map(Arranjo::tag).orElse("desconhecido");
    String statusTag  =
StatusEvento.parse(evento.status()).map(StatusEvento::tag).orElse("desconhecido");

    Resultado r;
    try {
        if (evento.enqueuedTime() != null)
            metrics.registrarEsperaFila(arranjoTag, Duration.between(evento.enqueuedTime(),
Instant.now()));
        if (evento.deliveryCount() != null && evento.deliveryCount() > 1)
            metrics.reentrega(arranjoTag);
        r = executar(evento, ...);          // valida, banco, máquina de estado, publica
    } catch (RuntimeException e) {
        r = new Resultado(Outcome.ERROR, PagamentoMetrics.classificar(e), ...);
    }
    metrics.registrarProcessamento(sample, arranjoTag, statusTag, r.outcome(), r.errorType());
    return r;
}

// dentro de executar(): transição inválida
Optional<EstadoOperacao> novo = MaquinaDeEstado.transicionar(atual.estado(), status);
if (novo.isEmpty()) {
    metrics.medirDb(OperacaoDb.GRAVAR_OCORRENCIA, () -> repository.gravarOcorrencia(...));
    metrics.transicaoInvalida(arranjo.tag(), atual.estado().tag(), status.tag());
    return new Resultado(Outcome.REJECTED, ErrorType.TRANSICAO_INVALIDA, ...);
}
```

4.4 Configuração (`application.yml`)

```
management:
  endpoints.web.exposure.include: health,info,prometheus,metrics
  metrics:
    tags:
      application: ${spring.application.name}
      env: ${AMBIENTE:local}
    distribution:
      slo:
        "[pagamento.evento.processamento]": 10ms,25ms,50ms,100ms,250ms,500ms,1s,2s,5s
        "[pagamento.evento.espera_fila]":   10ms,50ms,100ms,500ms,1s,5s,30s,60s
        "[pagamento.db.operacao]":          5ms,10ms,25ms,50ms,100ms,250ms,1s
        "[pagamento.servicebus.envio]":     10ms,25ms,50ms,100ms,250ms,500ms,1s,2s
      minimum-expected-value."[pagamento]": 1ms
      maximum-expected-value."[pagamento]": 60s
```

Os buckets devem incluir o valor do SLO (aqui 250ms para processamento). Assim, “% dentro do SLO” é lido direto do bucket `le="0.25"`, sem estimar percentil. Ajustar os valores conforme o SLO acordado com o negócio.

4.5 Testes

Testes unitários com `SimpleMeterRegistry`, sem latência nem falhas injetadas, cobrindo:

- Fluxo feliz PIX (4 eventos): `processamento{outcome=success}` = 4, `servicebus.envio{destino=pagamento-concluido}` = 1, `db.operacao{operacao=atualizar_status}` = 4, `espera_fila` = 4.

- Transição inválida (TED `iniciada → pagamento_devolvido`): counter com as três tags, `gravar_ocorrencia` = 1, `processamento{outcome=rejected, error.type=transicao_invalida}` = 1.
- Não mapeados: arranjo `DOC` → `arranjo=desconhecido, motivo=arranjo_desconhecido`; status `PAGAMENTO_AGENDADO` → `status_desconhecido`; antifraude em boleto → `fluxo_nao_suportado`.
- Erro técnico do Service Bus: `servicebus.envio{outcome=error, error.type=servicebus}` e `processamento{outcome=error, error.type=servicebus}`.
- Reentrega: `deliveryCount = 3` incrementa `reentrega{arranjo}`.

4.6 Estimativa de cardinalidade

Métrica	Combinações máximas	Séries observadas na POC	Com buckets
`processamento`	7 arranjos × 9 status × 3 outcomes × 8 erros (teórico); na prática bem menos, porque `error.type` depende de `outcome`	~80	~800 (+3 por série: count, sum, max)
`servicebus.envio`	6 destinos × 7 arranjos × combinações de outcome/erro	~40	~350
`db.operacao`	3 operações × outcome/erro	~8	~64
`transicao.invalida`	7 × 9 × 9, limitado pelas transições que ocorrem	~25	—

Total abaixo de 1.500 séries por pod. Se a cardinalidade crescer, a primeira medida é remover `error.type` do histograma, mantendo-o só no `_count` via um counter dedicado.

5. Cenários de monitoramento

O que cada métrica permite enxergar, com a query de referência e a ação esperada. Todos os sinais têm painel correspondente no dashboard (seção 7).

Cenário	Sinal	Query (PromQL)	Ação esperada
Queda ou pico de vazão	Eventos/s total e por arranjo fora do padrão	`sum by (arranjo) (rate(pagamento_evento_processamento_seconds_count[5m]))`	Comparar com a mesma janela da semana anterior; queda a zero indica produtor parado ou consumidor travado.
Aumento de erro técnico	% `outcome=error` sobe	`sum(rate(..._count{outcome="error"}[5m])) / sum(rate(..._count[5m]))`	Abrir por `error_type` para identificar a dependência.
Erro concentrado em um arranjo	Taxa de erro de um arranjo destoa dos demais	`sum by (arranjo) (rate(..._count{outcome="error"}[5m])) / sum by (arranjo) (rate(..._count[5m]))`	Verificar regra ou integração específica do arranjo.

Banco lento	p95 de `db.operacao` sobe e puxa o p95 do processamento	`histogram_quantile(0.95, sum by (le, operacao) (rate(pagamento_db_operacao_seconds_bucket[5m])))`	Identificar a operação (busca, atualização ou ocorrência); verificar índices e pool.
Banco falhando	`db.operacao{outcome=error}` > 0	`sum by (operacao, error_type) (rate(pagamento_db_operacao_seconds_count{outcome="error"}[5m]))`	Separar `db` de `timeout`; acionar time de dados se persistir.
Fila de saída falhando	Falhas de envio por `destino`	`sum by (destino, error_type) (rate(pagamento_servicebus_envio_seconds_count{outcome="error"}[5m]))`	Checar namespace, quota e permissões da fila específica.
Fila de saída lenta	p95 de envio por destino sobe	`histogram_quantile(0.95, sum by (le, destino) (rate(pagamento_servicebus_envio_seconds_bucket[5m])))`	Verificar throttling do Service Bus.
Timeouts em dependências	`error_type="timeout"` em banco ou envio; `_max` perto do timeout configurado	`sum by (error_type) (rate(pagamento_evento_processamento_seconds_count{error_type="timeout"}[5m]))`	Revisar timeouts e retry; correlacionar com a dependência.
Backlog na fila de entrada	Espera na fila p95 sobe enquanto o processamento segue normal	`histogram_quantile(0.95, sum by (le, arranjo) (rate(pagamento_evento_espera_fila_seconds_bucket[5m])))`	Escalar consumidores ou aumentar `maxConcurrentCalls`.
Risco de DLQ	Reentregas por arranjo sobem	`sum by (arranjo) (rate(pagamento_evento_reentrega_total[5m]))`	Investigar o erro técnico que causa *abandon* antes que as mensagens caiam na DLQ.
Eventos duplicados ou fora de ordem	Surto de `transicao_invalida` com `estado_atual = destino` do status (duplicado) ou estado anterior ao esperado (fora de ordem)	`sum by (estado_atual, status_evento) (increase(pagamento_transicao_invalida_total[10m]))`	Duplicado: verificar idempotência do produtor. Fora de ordem: avaliar sessões do Service Bus por operação.
Transição nova não prevista	Par `estado_atual → status_evento` que nunca aparecia	`sum by (arranjo, estado_atual, status_evento) (increase(pagamento_transicao_invalida_total[$__range]))`	Validar com negócio se a máquina de estado precisa de nova regra.
Status novo enviado pelo upstream	`nao_mapeado{motivo=status_desconhecido}` > 0	`sum by (arranjo, motivo) (increase(pagamento_evento_nao_mapeado_total[10m]))`	Contrato mudou: mapear o status ou alinhar com o produtor.
Arranjo novo ou inválido	`motivo=arranjo_desconhecido`	Mesma query, filtrando `motivo`	Mapear o novo arranjo antes de liberar o tráfego.

Fluxo não suportado	`motivo=fluxo_nao_suportado` (ex.: antifraude em TED ou boleto)	Mesma query, filtrando `motivo`	Erro de roteamento no produtor.
Mudança no mix de status	Proporção de `fraude_reprovada`, `pagamento_rejeitado` ou `cancelado` sobe	`sum by (status_evento) (rate(..._count{outcome="success"}[5m])) / ignoring(status_evento) group_left sum(rate(..._count{outcome="success"}[5m]))`	Sinal de negócio (fraude, recusa da instituição): avisar o produto.
Quebra de SLO de latência	% de eventos ≤ 250ms abaixo da meta	`sum(rate(..._bucket{le="0.25"}[5m])) / sum(rate(..._count[5m]))`	Consumo de error budget; priorizar correção.
Cauda de latência	p99 ou `_max` alto com p50 estável	`max(max_over_time(pagamento_evento_processamento_seconds_max[5m]))`	Investigar timeouts pontuais, GC e pausas.
Etapa lenta por status	Tempo médio de um status específico sobe	`sum by (status_evento) (rate(..._sum[5m])) / sum by (status_evento) (rate(..._count[5m]))`	Identificar a etapa mais cara (ex.: conclusão com mais escrita).
Falha ao gravar ocorrência	`db.operacao{operacao=gravar_ocorrencia, outcome=error}`	`sum(rate(pagamento_db_operacao_seconds_count{operacao="gravar_ocorrencia",outcome="error"}[5m]))`	Crítico: a ocorrência pode se perder, então tratar como incidente.

`..._count`, `..._bucket` e `..._sum` referem-se a `pagamento_evento_processamento_seconds_*`. No Grafana, troque `[5m]` por `[$__rate_interval]` e aplique os filtros `{env=~"$env", arranjo=~"$arranjo", status_evento=~"$status_evento"}`.

6. Catálogo de queries

Vazão e resultado

```
# eventos/s total
sum(rate(pagamento_evento_processamento_seconds_count{env=~"$env", arranjo=~"$arranjo",
status_evento=~"$status_evento"}[$__rate_interval]))

# eventos/s por arranjo | por status | por outcome
sum by (arranjo)       (rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))
sum by (status_evento) (rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))
sum by (outcome)       (rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))

# % sucesso / % rejeitado / % erro técnico
sum(rate(pagamento_evento_processamento_seconds_count{..., outcome="success"}[$__rate_interval]))
  / sum(rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))

# não-sucesso por tipo de erro
sum by (error_type) (rate(pagamento_evento_processamento_seconds_count{..., outcome!="success"}
[$__rate_interval]))

# taxa de erro técnico por arranjo
sum by (arranjo) (rate(pagamento_evento_processamento_seconds_count{..., outcome="error"}
[$__rate_interval]))
  / sum by (arranjo) (rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))

# tabela: volume no período por arranjo × status × resultado
sum by (arranjo, status_evento, outcome)
(increase(pagamento_evento_processamento_seconds_count{...}[$__range]))
```

Latência

```
# p50 / p95 / p99 (troque 0.95 pelo quantil desejado)
histogram_quantile(0.95, sum by (le) (rate(pagamento_evento_processamento_seconds_bucket{...}
[$__rate_interval])))

# p95 por arranjo
histogram_quantile(0.95, sum by (le, arranjo)
(rate(pagamento_evento_processamento_seconds_bucket{...}[$__rate_interval])))

# pior caso na janela (Timer._max)
max(max_over_time(pagamento_evento_processamento_seconds_max{...}[$__rate_interval]))

# tempo médio por status
sum by (status_evento) (rate(pagamento_evento_processamento_seconds_sum{...}[$__rate_interval]))
  / sum by (status_evento) (rate(pagamento_evento_processamento_seconds_count{...}
[$__rate_interval]))

# % dentro do SLO de 250ms (exato, lido do bucket)
sum(rate(pagamento_evento_processamento_seconds_bucket{..., le="0.25"}[$__rate_interval]))
  / sum(rate(pagamento_evento_processamento_seconds_count{...}[$__rate_interval]))

# heatmap (formato heatmap no Grafana)
sum by (le) (increase(pagamento_evento_processamento_seconds_bucket{...}[$__rate_interval]))

# espera na fila p95 por arranjo
histogram_quantile(0.95, sum by (le, arranjo)
(rate(pagamento_evento_espera_fila_seconds_bucket{...}[$__rate_interval])))
```

Banco e Service Bus

```
histogram_quantile(0.95, sum by (le, operacao)
(rate(pagamento_db_operacao_seconds_bucket{env=~"$env"}[$__rate_interval])))
sum by (operacao, outcome)    (rate(pagamento_db_operacao_seconds_count{env=~"$env"}
[$__rate_interval]))
sum by (operacao, error_type) (rate(pagamento_db_operacao_seconds_count{env=~"$env",
outcome="error"}[$__rate_interval]))

sum by (destino)             (rate(pagamento_servicebus_envio_seconds_count{...}
[$__rate_interval]))
sum by (destino, error_type) (rate(pagamento_servicebus_envio_seconds_count{..., outcome="error"}
[$__rate_interval]))
histogram_quantile(0.95, sum by (le, destino) (rate(pagamento_servicebus_envio_seconds_bucket{...}
[$__rate_interval])))
```

Máquina de estado, não mapeados e reentregas

```
# transições inválidas por minuto
sum by (estado_atual, status_evento) (rate(pagamento_transicao_invalida_total{...}
[$__rate_interval])) * 60

# matriz de transições inválidas no período (tabela)
sum by (arranjo, estado_atual, status_evento) (increase(pagamento_transicao_invalida_total{...}
[$__range]))

# não mapeados por motivo, por minuto
sum by (motivo) (rate(pagamento_evento_nao_mapeado_total{env=~"$env", arranjo=~"$arranjo"}
[$__rate_interval])) * 60

# reentregas por arranjo
sum by (arranjo) (rate(pagamento_evento_reentrega_total{env=~"$env", arranjo=~"$arranjo"}
[$__rate_interval]))
```

Variáveis do dashboard

```
env           = label_values(pagamento_evento_processamento_seconds_count, env)
arranjo       = label_values(pagamento_evento_processamento_seconds_count{env=~"$env"}, arranjo)
status_evento = label_values(pagamento_evento_processamento_seconds_count{env=~"$env",
arranjo=~"$arranjo"}, status_evento)
# todas multi-seleção, com "All" = .*
```

7. Dashboard

Um dashboard **“Pagamentos — fluxo de eventos”** com filtros de Ambiente, Arranjo e Status do evento no topo e seis linhas:

Linha	Painéis	Pergunta que responde
Visão geral	Stats: eventos/s, % sucesso, % rejeitado, % erro técnico, p95, % dentro do SLO	Está tudo bem agora?
Vazão e resultado por tag	Eventos/s por arranjo, por status, por outcome; não-sucesso por `error_type`; taxa de erro por arranjo; tabela arranjo × status × resultado	De onde vem o volume e onde está o erro?

Latência	p50/p95/p99 com max no eixo direito; p95 por arranjo; tempo médio por status; espera na fila p95; heatmap dos buckets	Está lento? Para quem? É processamento ou fila?
Banco de dados	p95 por operação; ops/s por operação e resultado; erros por tipo	O banco é o gargalo?
Service Bus	Envios/s por destino; falhas por destino e tipo; p95 por destino	Alguma fila de saída está falhando?
Máquina de estado	Transições inválidas/min (`estado_atual → status_evento`); matriz de transições no período; não mapeados por motivo; reentregas por arranjo	Que inconsistência de ordem ou estado está acontecendo?

Cores fixas para `outcome` : success verde, rejected amarelo, error vermelho. O dashboard deve ser versionado como JSON no repositório e provisionado no Grafana.

8. Alertas sugeridos

Alerta	Expressão	Severidade
TaxaErroTecnicoAlta	`sum by (arranjo) (rate(..._count{outcome="error"}[5m])) / sum by (arranjo)(rate(..._count[5m])) > 0.05` por 5m	critical
LatenciaProcessamentoP95Alta	`histogram_quantile(0.95, sum by (le, arranjo) (rate(..._bucket[5m]))) > 0.5` por 10m	warning
BacklogNaFila	`histogram_quantile(0.95, sum by (le, arranjo) (rate(pagamento_evento_espera_fila_seconds_bucket[5m]))) > 5` por 5m	warning
FalhaEnvioServiceBus	`sum by (destino) (rate(pagamento_servicebus_envio_seconds_count{outcome="error"}[5m])) > 0.1` por 5m	critical
FalhaGravarOcorrencia	`sum(increase(pagamento_db_operacao_seconds_count{operacao="gravar_ocorrencia",outcome="error"}[5m])) > 0`	critical
SurtoTransicaoInvalida	`sum by (arranjo, estado_atual, status_evento) (increase(pagamento_transicao_invalida_total[10m])) > 50`	warning
EventosNaoMapeados	`sum by (motivo) (increase(pagamento_evento_nao_mapeado_total[10m])) > 0`	info
ReentregasAltas	`sum by (arranjo) (rate(pagamento_evento_reentrega_total[5m])) > 1` por 10m	warning

Limiares são ponto de partida. Calibrar com 1 a 2 semanas de dados de produção. Definição formal de SLO e roteamento de alertas ficam para a próxima história.

9. Critérios de aceite

1. As sete métricas do catálogo estão expostas em `/actuator/prometheus` com os nomes, tipos e tags definidos na seção 3.
2. Toda métrica publica sempre o mesmo conjunto de chaves de tag. Valores fora do domínio aparecem como `desconhecido`, e ausência de erro aparece como `none`.
3. Os valores de tag vêm de enums centralizados (`Tags` e enums de domínio). Nenhuma tag contém identificador de operação, dado de cliente ou mensagem de exceção.
4. O código de negócio não acessa `MeterRegistry` diretamente: toda instrumentação passa por `PagamentoMetrics`.
5. `pagamento.evento.processamento` é registrado exatamente uma vez por mensagem, em todos os caminhos (sucesso, rejeição e exceção), com `outcome` e `error.type` corretos.
6. Dado um evento cujo status não é transição válida a partir do estado salvo, quando ele é processado, então a ocorrência é gravada, `pagamento.transicao.invalida` é incrementado com `arranjo`, `estado_atual` e `status_evento`, e o processamento termina com `outcome=rejected` e `error.type=transicao_invalida`.
7. Dado um evento com status ou arranjo desconhecido, ou um status que não existe no fluxo do arranjo, quando ele é processado, então `pagamento.evento.nao_mapeado` é incrementado com o `motivo` correspondente e não há chamada ao banco.
8. `pagamento.evento.espera_fila` é calculado a partir do `enqueuedTime` da mensagem do Service Bus. Valores negativos (clock skew) são registrados como zero.
9. Mensagens com `deliveryCount > 1` incrementam `pagamento.evento.reentrega`.
10. Falhas de banco e de envio são medidas por operação e por destino, com `error.type` categorizado (`db`, `servicebus`, `timeout`, `desconhecido`) e a exceção original é propagada sem alteração.
11. Os timers têm SLO buckets configurados em `application.yml`, incluindo o valor do SLO. Não são usados percentis no cliente nem `publishPercentileHistogram`.
12. As tags comuns `application` e `env` são aplicadas via configuração.
13. Testes unitários com `SimpleMeterRegistry` cobrem os cenários da seção 4.5.
14. O dashboard do Grafana (seção 7) está versionado como JSON no repositório e todas as queries retornam dados em ambiente de homologação.
15. As regras de alerta (seção 8) estão versionadas e carregam sem erro no Prometheus.

10. Subtarefas sugeridas

1. Criar `Tags` e os enums de valores; adicionar `tag()` / `parse()` nos enums de domínio.
2. Criar `PagamentoMetrics` com os sete meters e `classificar(Throwable)`; mapear as exceções reais (driver do banco, SDK do Service Bus, timeouts) para `ErrorType`.
3. Instrumentar o consumidor: `enqueuedTime`, `deliveryCount` e timer ponta a ponta com ponto único de fechamento.
4. Envolver as chamadas de banco com `medirDb` (busca, atualização, ocorrência).
5. Envolver os envios com `medirEnvio`, usando nome lógico da fila como `destino`.
6. Instrumentar a máquina de estado (transição inválida) e a validação de domínio (não mapeados).
7. Configurar `application.yml`: exposição do endpoint, tags comuns, SLO buckets.
8. Escrever os testes unitários.
9. Versionar e provisionar o dashboard e as regras de alerta; validar em homologação.

10. Documentar no PR a estimativa de cardinalidade observada.

11. Fora de escopo

- Definição formal de SLOs, error budget e roteamento de alertas (on-call): próxima história.
- Tracing distribuído. A instrumentação pode migrar para a Observation API (`Observation` / `@Observed`) depois, ganhando métrica e trace com o mesmo instrumento.
- Métricas de infraestrutura (JVM, pool de conexões, driver do banco), que já vêm da auto-configuração do Spring Boot Actuator.

12. Definition of Done

- Critérios de aceite atendidos e testes passando no pipeline.
- Estimativa de cardinalidade documentada no PR.
- Dashboard e alertas revisados com o time e validados em homologação com tráfego real.
- README do serviço atualizado com o catálogo de métricas.

Anexo — Ambiente de referência

A POC `pagamentos-metricas` implementa toda esta história com entrada HTTP no lugar do Service Bus, banco e filas simulados, e um gerador de tráfego com os cenários da seção 5. Serve como referência de código e para validar dashboard e alertas localmente.

```
docker compose up -d --build
# App:        http://localhost:8080   (POST /eventos, /actuator/prometheus)
# Prometheus: http://localhost:9090   (aba Alerts com as regras)
# Grafana:    http://localhost:3000   (dashboard provisionado como home)

# simular degradação do Service Bus (20% de erro)
curl -X PUT localhost:8080/chaos -H 'Content-Type: application/json' \
  -d
'{"dbLatenciaMs":15,"sbLatenciaMs":25,"dbTaxaErro":0.01,"sbTaxaErro":0.2,"taxaTimeout":0.003,"timeo
utMs":1500}'

# simular backlog de 3s na fila de entrada
curl -X PUT 'localhost:8080/simulador?eventosPorSegundo=80&atrasoFilaMs=3000'
```